

Dockerfile Security Best Practices

From Insecure to Invincible

Why Does This Matter? (The "So What?")

The Risk

Insecure images lead to:

- Root access vulnerabilities
- Leaked secrets and credentials
- Bloated, slow images
- Huge attack surfaces

The Reward

Secure images are:

- **Small:** Faster to pull, less to scan.
- **Safe:** Run with minimal privileges.
- **Lean:** Contain **only** what's needed.
- **Compliant:** Pass security audits.

Our Demo: The "Before" (A "Bad" Dockerfile)

The Problems

1. Uses `latest` tag (unpredictable).
2. Runs as ****ROOT**** by default.
3. Copies **all** files (source, `.git`).
4. Leaves build tools in final image.
5. Is HUGE (e.g., 900MB+).

Dockerfile.bad

```
FROM python:latest
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["flask", "run", "--host=0.0.0.0"]
```


BP #1: Use a Minimal Base Image

The Why

Don't use `latest`, `python:3.10`, or `ubuntu`. They are full operating systems with tools (`curl`, `bash`) you don't need, which an attacker can use.

The Solution

Use `slim`, `alpine`, or `distroless` images.

```
# Good
FROM python:3.11-slim

# Better
FROM python:3.11-alpine

# Best (Security)
FROM gcr.io/distroless/python3
```


BP #2: Don't Run as Root

The Why

This is the #1 rule. Root in the container is root on the host (if it escapes). Limiting privileges is your best defense against container breakout.

The Code

```
# Create a user
RUN addgroup -S appgroup && \
    adduser -S appuser -G appgroup

# ...
# Do all your work as root...
# ...

# Switch to the new user AT THE END
USER appuser
```


BP #3: Use Multi-Stage Builds

The Why

Separate your **build** environment from your **runtime** environment. The final image should ****NOT**** contain compilers, SDKs, or your source code.

The Concept

```
# Stage 1: The "Builder"
FROM python:3.11-slim AS builder
WORKDIR /app
RUN pip install ...

# Stage 2: The "Final" Image
FROM gcr.io/distroless/python3
WORKDIR /app
COPY --from=builder /app /app
# ...
```


BP #4: Minimize Attack Surface

The Why

Every file you `COPY` into the image is a potential risk or bloat. Be explicit. Don't copy secrets, `.git` folders, or local development files.

The How

1. **Use `.dockerignore`**: Add `.git`, `*.pyc`, `__pycache__` to it.
2. **Be specific**: `COPY requirements.txt .` not `COPY ..`
3. **Clear cache**: Add `RUN rm -rf /var/lib/apt/lists/*`

BP #5: Handle Secrets Securely

The Why

NEVER copy secrets (like `API\_KEY`) into a Dockerfile. They will be baked into the image layers, visible to anyone who runs `docker history`.

The Solution

Build-time: Use BuildKit secret mounts.

```
RUN --mount=type=secret,id=mysecret \  
    cat /run/secrets/mysecret
```

Run-time: Use environment variables (`-e`) or Docker /

BP #6: Scan Your Images!

The Why

You wouldn't ship code without testing. Don't ship an image without scanning. Find known vulnerabilities **before** they hit production.

The Commands

```
# Natively in Docker
docker scan my-app:latest

# Using open-source Trivy
# (Highly Recommended)
trivy image my-app:latest
```


Our Demo: The "After" (A "Good" Dockerfile)

The Wins

1. Uses a minimal `distroless` base.
2. Runs as a ****non-root user****.
3. Uses ****multi-stage build****.
4. No source code or tools in final image.
5. Is TINY (e.g., 90MB vs 900MB).

Dockerfile.good

```
FROM python:3.11-slim AS builder
RUN pip install ...

FROM gcr.io/distroless/python3
COPY --from=builder /app /app
RUN adduser -S appuser ...
USER appuser
CMD ["my-app"]
```


Key Takeaways

-  **Use Multi-Stage Builds:** Separate build from runtime.
-  **Run as Non-Root:** Create a dedicated, unprivileged user.
-  **Use Minimal Base Images:** Start with ``alpine`` or ``distroless``.
-  **Scan Your Images:** Use ``trivy`` or ``docker scan`` in CI/CD.
-  **Use ``.dockerignore``:** Be explicit about what you ``COPY``.

Questions?

Thank you! All code is in the description.

@YourSocialHandle